

UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS (CUCEI)

División de Electrónica y Computación

Actividad 1.2: Carga de Memoria Distribuida en MPI y OpenMP

Materia: Programación Paralela y Distribuida

Integrantes del Equipo (Orden Alfabético por Primer Apellido):

1. ANGUIANO GARCIA, ANGEL YAHIR GUADALUPE

2. FIGUEROA ROBLES, AXEL ISRAEL

Fecha: Septiembre de 2026

1. Resumen y Enlaces de Entrega

- **Enlace Público del Video (YouTube):** *[Pega aquí el enlace de tu video de YouTube]*
- **Enlace Público del Repositorio:** *[Pega aquí el enlace de GitHub]*

2. Objetivo de la Práctica

Implementar un modelo híbrido de paralelismo combinando **Memoria Distribuida (MPI)** y **Memoria Compartida (OpenMP)**. El procesamiento se divide entre procesos mediante un esquema Maestro-Trabajador (1 nodo maestro y 4 nodos trabajadores, 5 procesos en total), mientras que dentro de cada proceso se emplea OpenMP para paralelizar las operaciones aritméticas a nivel de hilos utilizando los núcleos disponibles en la CPU sobre arreglos dinámicos mediante punteros tradicionales (double*).

3. Explicación de la Clase OperacionesArreglos

La clase `OperacionesArreglos` encapsula las operaciones aritméticas elementales sobre los arreglos dinámicos:

- **crearArregloMPI:** Inicializa las secciones de los arreglos. En modo detallado (40 elementos) genera números consecutivos; en modo masivo (4,000,000 elementos) genera valores aleatorios entre 1 y 1,000,000 mediante un generador independiente por hilo.
- **sumar:** Calcula $C[i] = A[i] + B[i]$ mediante `#pragma omp parallel for`.
- **restar:** Calcula $C[i] = A[i] - B[i]$ mediante `#pragma omp parallel for`.
- **multiplicar:** Calcula $C[i] = A[i] * B[i]$ mediante `#pragma omp parallel for`.
- **cuadrado:** Calcula $C[i] = (A[i])^2$ mediante `#pragma omp parallel for`.

4. Explicación de la Distribución MPI

La solución implementa el patrón Maestro-Trabajador con 5 procesos (Rank 0 como Maestro, Ranks 1 al 4 como Trabajadores):

- **Nodo Maestro (Rank 0):** Muestra el menú, transmite la opción con `MPI_Bcast`, distribuye las secciones a los trabajadores, recopila los resultados y mide el tiempo de ejecución con `MPI_Wtime()`.
- **Nodos Trabajadores (Ranks 1 a 4):** Cada uno procesa un bloque equitativo de tamaño $N/4$ (10 elementos en la prueba de 40, y 1,000,000 de elementos en la prueba de 4M).

5. Comparación entre Implementaciones MPI

Criterio	Versión 1: MPI_Send / MPI_Recv	Versión 2: MPI_Scatter / MPI_Gather
Tipo de Comunicación	Punto a punto individualizada.	Colectiva optimizada (Uno a Todos / Todos a Uno).
Estructura del Código	Requiere bucles for iterativos para enviar a cada nodo trabajador.	Una sola instrucción colectiva que sincroniza a todos los nodos.
Sincronización	Bloqueante por mensaje según buffers de red.	Sincronización colectiva implícita.
Rendimiento	Mayor sobrecarga de mensajes individuales.	Altamente optimizado en la pila interna de MPI.

6. Resultados y Tiempos de Ejecución Obtenidos (4,000,000 de Elementos)

Operación	Tiempo Versión 1 (Send / Recv)	Tiempo Versión 2 (Scatter / Gather)
Inicialización / Creación	0.036815 segundos	0.076443 segundos
Suma (A + B)	0.032283 segundos	0.072024 segundos
Resta (A - B)	0.031340 segundos	0.052274 segundos
Multiplicación (A * B)	0.036122 segundos	0.051101 segundos
Cuadrado (A^2)	0.020414 segundos	0.034866 segundos

7. Evidencias de Ejecución (Salidas de Terminal)

Formato estandarizado de mensaje obligatorio:

```
[Equipo: AxelFxd] [Proceso MPI: 1] [Hilo OpenMP: 0] [Posición: 0] [Operación:
Cuadrado -> (1)^2 = 1]
[Equipo: AxelFxd] [Proceso MPI: 1] [Hilo OpenMP: 0] [Posición: 1] [Operación:
Cuadrado -> (2)^2 = 4]
[Equipo: AxelFxd] [Proceso MPI: 1] [Hilo OpenMP: 1] [Posición: 2] [Operación:
Cuadrado -> (3)^2 = 9]
[Equipo: AxelFxd] [Proceso MPI: 2] [Hilo OpenMP: 0] [Posición: 10] [Operación:
Cuadrado -> (11)^2 = 121]
[Equipo: AxelFxd] [Proceso MPI: 2] [Hilo OpenMP: 1] [Posición: 12] [Operación:
Cuadrado -> (13)^2 = 169]
[Equipo: AxelFxd] [Proceso MPI: 3] [Hilo OpenMP: 0] [Posición: 20] [Operación:
Cuadrado -> (21)^2 = 441]
[Equipo: AxelFxd] [Proceso MPI: 3] [Hilo OpenMP: 1] [Posición: 22] [Operación:
Cuadrado -> (23)^2 = 529]
[Equipo: AxelFxd] [Proceso MPI: 4] [Hilo OpenMP: 0] [Posición: 30] [Operación:
Cuadrado -> (31)^2 = 961]
[Equipo: AxelFxd] [Proceso MPI: 4] [Hilo OpenMP: 7] [Posición: 39] [Operación:
Cuadrado -> (40)^2 = 1600]
[Equipo Maestro: AxelFxd] [Proceso MPI: 0] Enviando sección (offset 0, tam 10) al
Trabajador MPI: 1
[Equipo Maestro: AxelFxd] [Proceso MPI: 0] Enviando sección (offset 10, tam 10) al
Trabajador MPI: 2
[Equipo Maestro: AxelFxd] [Proceso MPI: 0] Enviando sección (offset 20, tam 10) al
Trabajador MPI: 3
```

```
[Equipo Maestro: AxelFxd] [Proceso MPI: 0] Enviando sección (offset 30, tam 10) al  
Trabajador MPI: 4
```

8. Conclusiones

La combinación de MPI para memoria distribuida y OpenMP para multinúcleo en memoria compartida permitió explotar de forma óptima las capacidades de hardware. La asignación dinámica de memoria mediante punteros tradicionales aseguró la contigüidad física necesaria para la transmisión eficiente de buffers en MPI, logrando procesar 4 millones de elementos en milisegundos con exactitud numérica total.